实验一: Threads and synchronize

Note: 你可以直接使用 Linux 上的 gcc 完成本次作业；不需要 xv6 环境。

说明：

- 本实验在任何一台 Linux 上使用 gcc 即可完成，包括虚拟机中的 Linux ，无平台限制，不需要像其他一些实验一样需要 xv6 环境。
- 虚拟机可以使用VMware Workstation或Oracle VirtualBox，也可以使用Microsoft wsl(windows subsystem for linux)，注意本实验需要为虚拟机分配多于1个的CPU核，以期观察多核环境下的竞争和同步现象。

学习目标：

理解多线程并发中的竞态条件：通过哈希表实验观察到，单线程不会丢 key，但多个线程同时执行 `put/get` 时会因为并发访问共享数据而出现错误，从而理解为什么需要同步机制

掌握互斥锁的基本使用：学会使用 `pthread_mutex_init`、`pthread_mutex_lock`、`pthread_mutex_unlock` 来保护共享数据，消除数据竞争，保证程序正确性

理解“正确性”和“性能”之间的关系：不仅要加锁修复错误，还要思考锁是否过多导致并行度下降，例如让 `get` 尽量并行、进一步尝试按 bucket 加锁来提升 `put` 的并行性

学会分析并发程序的执行过程：实验要求学生找出一个会导致 key 丢失的线程交错执行序列，这有助于建立对线程调度、共享状态和错误产生原因的深入理解

掌握条件变量与屏障同步：在第二部分中，学生要使用 `pthread_cond_wait` 和 `pthread_cond_broadcast` 实现 barrier，理解“所有线程都到达某一点后才能继续”的同步模式

理解更复杂的同步问题：例如 barrier 的“多轮次”问题，以及一个线程过快进入下一轮时如何避免破坏上一轮的同步状态，从而学习如何设计更严谨的并发控制逻辑

Part One: Locking

在这部分作业中，你将通过一个哈希表来探索使用线程和锁进行并行编程。你应该在一台真实的多核计算机上完成这项作业（不是 xv6，也不是 qemu）。大多数现代笔记本电脑都具有多核处理器。

Download ph.c and compile it:

```
$ gcc -g -O2 ph.c -pthread
$ ./a.out 2
```

其中参数 `2` 表示有两个线程会对哈希表执行 `put` 和 `get` 操作。
程序运行一段时间后，会输出类似如下内容：

```
1: put time = 0.003338
0: put time = 0.003389
0: get time = 7.684335
0: 17480 keys missing
1: get time = 7.684335
1: 17480 keys missing
completion time = 7.687856
```

每个线程分两个阶段运行。第一阶段中，每个线程向哈希表插入 `NKEYS/nthread` 个键。第二阶段中，每个线程从哈希表中查找 `NKEYS` 个键。输出中的打印信息告诉你每个线程在每个阶段花费了多少时间。最后一行的 `completion time` 表示整个应用程序的总运行时间。

在上面的输出中，程序总完成时间大约是 7.7 秒。每个线程大约运行了 7.7 秒（`put` 阶段约为 0.0 秒，`get` 阶段约为 7.7 秒）。

为了判断使用两个线程是否提升了性能，我们来和单线程情况进行比较：

```
$ ./a.out 1
0: put time = 0.004073
0: get time = 6.929189
0: 0 keys missing
completion time = 6.933433
```

单线程情况下，总完成时间约为 7.0 秒，略少于双线程情况下的约 7.7 秒；但双线程在 `get` 阶段完成的总工作量是单线程的两倍。因此，在双核上，双线程在 `get` 阶段实际上获得了接近 2 倍的并行加速，这非常不错。

当你运行这个程序时，如果你的机器只有一个核心，或者机器正在忙于运行其他程序，你可能观察不到并行性。

这里有两点需要注意：

1. 程序总完成时间与双线程运行时差不多，但这次运行完成的 `get` 操作数量是双线程情况的两倍；这说明我们获得了良好的并行性。
2. 双线程输出显示有很多键丢失了。你自己运行时，丢失的键数量可能更多，也可能更少。如果你只运行 1 个线程，则永远不会有键丢失。

为什么在 2 个或更多线程时会出现键丢失，而在 1 个线程时不会？请找出一种会导致 2 个线程情况下键丢失的事件执行序列。

为了避免这种情况，请在 `put` 和 `get` 中插入加锁和解锁语句，使得丢失的键数始终为 0。相关的 pthread 调用如下（更多内容请参考手册页 `man pthread`）：

```
pthread_mutex_t lock;     // declare a lock
pthread_mutex_init(&lock, NULL);   // initialize the lock
pthread_mutex_lock(&lock);  // acquire lock
pthread_mutex_unlock(&lock);  // release lock
```

请先用 1 个线程测试你的代码，再用 2 个线程测试。它是否正确（即你是否消除了键丢失问题）？双线程版本是否比单线程版本更快？

修改你的代码，使 `get` 操作在保证正确性的前提下可以并行执行。（提示：在这个应用中，`get` 中的锁对正确性是否是必需的？）

进一步修改你的代码，使部分 `put` 操作在保证正确性的前提下也可以并行执行。（提示：是否可以为每个bucket设置一把锁？）

Part Two: Barriers

在这部分作业中，我们将探索如何使用 pthread 库提供的条件变量来实现一个屏障（barrier）。屏障是程序中的一个同步点：所有线程都必须等待，直到其他所有线程也到达该点后，才能继续执行。

条件变量是一种顺序协调技术，类似于 xv6 中的 `sleep` 和 `wakeup`。

下载 `barrier.c` 并编译：

```
$ gcc -g -O2 -pthread barrier.c
$ ./a.out 2
```

运行后你会看到类似错误：

```
Assertion failed: (i == t), function thread, file barrier.c, line 55.
```

其中参数 `2` 表示有两个线程会在屏障上同步（即 `barrier.c` 中的 `nthread`）。

每个线程都在一个紧密循环中运行。在每次循环迭代中，线程会调用 `barrier()`，然后随机休眠若干微秒。断言失败的原因是：某个线程在另一个线程尚未到达屏障之前，就提前离开了屏障。期望的行为是：所有线程都应当阻塞，直到有 `nthreads` 个线程都调用了 `barrier()`。

你的目标是实现这种期望行为。除了你之前见过的锁原语之外，你还需要以下新的 pthread 原语（详情请查看 `man pthreads`）：

```
pthread_cond_wait(&cond, &mutex);  // go to sleep on cond, releasing lock mutex
pthread_cond_broadcast(&cond);     // wake up every thread sleeping on cond
```

`pthread_cond_wait` 在调用时会释放互斥锁 `mutex`，并且在返回之前重新获取该锁。

我们已经为你提供了 `barrier_init()`。你的任务是实现 `barrier()`，使程序不再触发崩溃。`struct barrier` 已经定义好，其中的字段供你使用。

有两个问题会让这个任务更复杂：

1. 你必须处理一连串连续的屏障调用，我们将每一次调用称为一个“轮次（round）”。`bstate.round` 记录当前轮次。你应该在每一轮开始时增加 `bstate.round`。
2. 你必须处理这样一种情况：某个线程在其他线程尚未完全离开屏障时，就已经快速进入下一次循环。尤其要注意的是，你会在不同轮次之间重复使用 `bstate.nthread`。你必须确保：某个已经离开屏障并快速进入下一轮的线程，不会在前一轮仍在使用 `bstate.nthread` 时错误地增加它。

请使用 1 个线程、2 个线程以及多于 2 个线程进行测试。

提交内容：ph.c 、barrier.c、和实验报告，报告模板在智慧树上，提交到智慧树上